SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A COMPARISON OF AN AUTOMATED `KUBECTL`-BASED IMPERATIVE PROCEDURE AND ARGOCD GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
S.Kom.



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

HALAMAN PENGESAHAN

SKRIPSI

PERBANDINGAN PROSEDUR IMPERATIF TEROTOMASI BERBASIS `KUBECTL` DAN GITOPS DENGAN ARGOCD UNTUK MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Dr. techn. Kabul Kurniawan, S.Kom., M.Cs. Ketua Penguji
	Aina Musdholifah, S.Kom., M.Kom., Ph.D. Anggota Penguji

PERNYATAAN

Dengan ini saya menyatakan bahwa dalam Skripsi ini tidak terdapat karya yang pernah diajukan untuk memperoleh gelar kesarjanaan di suatu Perguruan Tinggi, dan sepanjang pengetahuan saya juga tidak terdapat karya atau pendapat yang ditulis atau diterbitkan oleh orang lain, kecuali yang secara tertulis diacu dalam naskah ini dan disebutkan dalam daftar pustaka.

Yogyakarta, 17 Juni 2026

Muhammad Argya Vityasy

PRAKATA

Isi Prakata di sini...

DAFTAR ISI

Halaman Judul **ii**

Halaman Pengesahan **iii**

Halaman Pernyataan **iv**

Halaman Motto **v**

Halaman Persembahan **vi**

PRAKATA **vii**

DAFTAR TABEL **x**

DAFTAR GAMBAR **xi**

DAFTAR LAMBANG DAN SINGKATAN **xiii**

INTISARI **xiv**

ABSTRACT **xv**

I PENDAHULUAN **1**

1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 4
1.6 Sistematika Penulisan 4

II TINJAUAN PUSTAKA **6**

2.1 Penyimpangan Konfigurasi (*Configuration Drift*) 6
2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi 6
2.2 Terminologi Dasar 7
2.2.1 Kubernetes dan Orkestrasi Kontainer 7
2.2.2 Git dan *Version Control* 7

2.2.3 *Continuous Integration* dan *Continuous Deployment* 8
2.2.4 *Infrastructure-as-Code* (IaC) 8
2.2.5 Paradigma Deklaratif vs Imperatif 8
2.3 Manajemen Konfigurasi Infrastruktur 8
2.4 *Continuous Deployment* dan Paradigma GitOps 9
2.4.1 Performa *Deployment* dan *Anti-Pattern* 9
2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif . 10
2.5.1 *Convergence* Menuju *Desired State* 10
2.5.2 *Audit Trail* sebagai *Social Control Mechanism* 10
2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan 11
2.7 Paradigma *Deployment* Alternatif 11
2.8 Evaluasi Empiris GitOps dan ArgoCD 11
2.9 Metode Eksperimental Terkontrol pada Riset Sistem 12
2.10 Klasifikasi Beban Uji (*Workload*) 13
2.11 Ruang Kontribusi Penelitian 14

III METODE PENELITIAN **15**
3.1 Deskripsi Umum dan Status Protokol 15
3.2 Rancangan Perbandingan 16
3.3 Alur Penelitian 17
3.4 Beban Uji dan Lingkungan 19
3.4.1 Gerbang Kesiapan 19
3.5 Intervensi dan Skenario 20
3.6 Definisi Variabel dan Metrik 22
3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan 22
3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan 23
3.6.3 R3: Batas Pengukuran 24
3.7 Protokol Pengulangan dan Penanganan Kegagalan 24
3.8 Pengulangan dan Analisis Data 26
3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan 27
3.9.1 Validitas Konstruk 27
3.9.2 Validitas Internal 27
3.9.3 Validitas Eksternal 28
3.9.4 Validitas Kesimpulan 28

IV HASIL DAN ANALISIS **29**
4.1 Status Data dan Provenans 29
4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1) 29
4.3 Format Pelaporan E2: Waktu (R2) 30
4.4 R3: Tidak Diukur 31
4.5 Rencana Analisis Komparatif 31
4.6 Batas Interpretasi 31

V KESIMPULAN DAN SARAN **33**
5.1 Kesimpulan 33
5.2 Saran 33
5.3 Keterbatasan Penelitian 34

A LAMPIRAN A **38**
1.1 Artefak Eksperimen yang Direncanakan 38
1.2 Skema Data dan Perlindungan Informasi 38
1.3 Status Lampiran 38

DAFTAR TABEL

3.1 Skenario intervensi yang direncanakan 21

4.1 Skema hasil R1 per skenario dan mode 30
4.2 Skema hasil R2 per skenario dan mode 30

DAFTAR GAMBAR

3.1 Rancangan dua mode pengelolaan berurutan pada satu stack Invenio-RDM; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah. 17
3.2 Bagan alir protokol dengan pembekuan sumber sebelum blok pengukuran dan tanpa surrogat pada desain formal. 18

DAFTAR LAMBANG DAN SINGKATAN

ArgoCD Alat GitOps *pull-based* yang melakukan *continuous reconciliation* terhadap *state* klaster Kubernetes

GitOps Paradigma manajemen *deployment* yang menggunakan repositori Git sebagai *single source of truth* dengan agen *reconciliation* otomatis

IaC *Infrastructure-as-Code*: praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi

R1 Metrik konsistensi *deployment*

R2 Metrik waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi

R3 Bukti keterlacakan; tidak diukur dalam studi ini karena audit API server tidak tersedia (lihat Bab V)

INTISARI

Perbandingan Prosedur Imperatif Terotomasi Berbasis `kubectl` dan GitOps dengan ArgoCD untuk Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy
23/522547/PA/22475

Skripsi ini merancang studi kasus komparatif terkontrol atas dua mode pengelolaan pada satu stack InvenioRDM produksi-like: mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD. Pada mode imperatif, *auto-sync*/*self-heal* ArgoCD dijeda, pemeriksa dan *runbook* otomatis menjalankan koreksi `kubectl`, lalu ArgoCD dipulihkan; ini bukan *deployment* terpisah dan bukan studi waktu reaksi manusia. Kedua mode berjalan berurutan pada stack yang sama; tidak ada stack kembar `exp-*` dan tidak ada eksekusi paralel A/B, sehingga non-isolasi terhadap beban latar dan dependensi bersama menjadi keterbatasan eksplisit. HPA normal dipertahankan pada kedua mode, sehingga skenario A menjadi kontrol HPA dan `/spec/replicas` dikecualikan dari R1 statis; skenario F diperlakukan terpisah sebagai batas visibilitas sumber daya di luar cakupan aplikasi; skenario G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. Metrik yang dirancang mencakup kesetaraan bidang konfigurasi dan kesehatan secara terpisah (R1), waktu deteksi, waktu ketersediaan bukti penyimpangan, dan waktu pemulihan konfigurasi penuh (R2). R3 tidak diukur karena audit API server tidak tersedia dan tidak dilaporkan sebagai nol atau tidak ada perbedaan. Sampel formal adalah $7 \times 2 \times 15 = 210$ run (10 R2 dan 5 R1 per sel) dengan sampling satu detik serta pembekuan revisi sumber, render, image digest, dan versi runtime. Protokol dan apparatus masih menunggu tinjauan serta validasi, dan data formal belum tersedia. Oleh sebab itu, skripsi ini belum melaporkan hasil komparatif atau kesimpulan empiris.

Kata Kunci: penyimpangan konfigurasi, Kubernetes, GitOps, otomasi imperatif, studi kasus komparatif terkontrol

ABSTRACT

A Comparison of an Automated `kubectl`-Based Imperative Procedure and ArgoCD GitOps for Addressing Configuration Drift in Kubernetes Clusters

By

Muhammad Argya Vityasy
23/522547/PA/22475

This thesis designs a controlled single-case comparative study of two management modes on one production-like InvenioRDM stack: an automated imperative mode based on `kubectl` and a GitOps mode based on ArgoCD. In the imperative mode, ArgoCD auto-sync/self-heal is paused, an automated checker and runbook perform `kubectl` corrections, and ArgoCD is then restored; this is not a separate deployment and not a human-reaction-time study. Both modes run sequentially on the same stack; there is no `exp-*` twin and no parallel A/B execution, so non-isolation from background load and shared dependencies is an explicit limitation. Normal HPA operation is retained in both modes, so scenario A is an HPA control and `/spec/replicas` is excluded from static R1; scenario F is a visibility-boundary case for resources outside the application scope; scenario G is a descriptive case that is not an ArgoCD self-heal case. The measures comprise configuration-field equality and health reported separately (R1), time to detection, availability of verifiable deviation evidence, and full configuration restoration (R2). R3 is not measured because Kubernetes API-server audit is unavailable and is not reported as zero or as no difference. The formal sample is 7 x 2 x 15 = 210 runs (10 R2 and 5 R1 per cell) with one-second sampling and frozen source revision, render, image digests, and runtime versions. The protocol and apparatus remain subject to review and validation, and no formal dataset is available. Accordingly, no comparative findings or empirical conclusions are reported in this draft.

Keywords: configuration drift, Kubernetes, GitOps, automated imperative management, controlled single-case comparative study

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes mengelola sumber daya aplikasi melalui keadaan yang dideklarasikan dan rekonsiliasi oleh komponen pengendali. Dalam operasi nyata, keadaan aktual dapat berbeda dari deklarasi yang menjadi acuan. Perbedaan ini disebut penyimpangan konfigurasi (*configuration drift*). Penyimpangan perlu dibedakan dari cacat statis: suatu manifest dapat mengandung konfigurasi yang keliru tetapi tetap sama dengan keadaan aktual, sedangkan drift merujuk pada ketidakcocokan antara acuan dan keadaan yang berjalan. Dengan demikian, pengukuran kesetaraan konfigurasi tidak dengan sendirinya menilai apakah nilai konfigurasi tersebut baik atau aman (Zhang et al., 2026; Rahman et al., 2023).

Pengelolaan deklaratif dapat menggunakan pengendali GitOps untuk membandingkan sumber daya klaster dengan deklarasi di repositori dan mencoba merekonsiliasikannya. Pendekatan imperatif menggunakan perintah untuk mengubah keadaan klaster; perilaku setelah perubahan bergantung pada otomasi dan prosedur yang dipasang. Karena itu, perbandingan empiris perlu menetapkan dengan jelas mekanisme yang dibandingkan. Penelitian ini tidak mengukur kecepatan atau keahlian manusia. Mode pembanding imperatif menggunakan prosedur otomatis berbasis `kubectl` yang tetap dan terstandarisasi, sedangkan mode GitOps menggunakan ArgoCD dengan kebijakan rekonsiliasi yang diverifikasi sebelum eksperimen. Kedua mode dijalankan berurutan pada satu stack InvenioRDM produksi-like; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah.

Sejumlah penelitian telah mengevaluasi GitOps maupun membandingkannya dengan pendekatan lain. Shrestha dan Ali membahas waktu induksi, penemuan, dan pemulihan drift dalam perbandingan GitOps dengan Ansible (Shrestha and Ali, 2024). Kagganti Nataraja mendeskripsikan perbandingan terkontrol pada Minikube satu node dengan konfigurasi keamanan yang berbeda (Nataraja, 2025). Matubber juga memasukkan pembanding lama yang semi-terotomasi dalam evaluasi infrastruktur Kubernetes (Matubber, 2025). Studi-studi tersebut menunjukkan bahwa evaluasi komparatif telah dilakukan; penelitian ini tidak mengklaim sebagai perbandingan pertama atau menganggap bahwa GitOps pasti lebih unggul. Rincian lingkup dan batas

interpretasi studi tersebut dibahas pada Bab II.

Penelitian ini merancang pengukuran per skenario pada beban uji multi-service InvenioRDM. Pengukuran membedakan kesetaraan bidang konfigurasi dari kesehatan layanan, memisahkan waktu deteksi dari ketersediaan bukti yang dapat diverifikasi, dan mengukur pemulihan terhadap kesetaraan inventaris serta bidang yang telah ditetapkan. Skenario A diperlakukan sebagai kontrol HPA, F sebagai batas visibilitas sumber daya di luar cakupan aplikasi, dan G sebagai kasus deskriptif yang bukan *self-heal* ArgoCD; ketiganya tidak dipool ke estimasi efikasi B–E. Keterlacakan (R3) tidak diukur karena audit API server Kubernetes tidak tersedia; keterbatasan ini dibahas pada Bab III dan Bab V.

Rancangan ini dimaksudkan untuk memperjelas perbedaan operasional antara dua prosedur yang ditetapkan pada konteks eksperimen tertentu. Karena kedua mode memakai satu stack secara berurutan, perlakuan tidak terisolasi dari beban latar dan dependensi bersama. Rekomendasi praktis maupun kesimpulan tentang arah perbedaan hanya dapat dibuat setelah apparatus divalidasi, protokol ditinjau, dan data formal dikumpulkan. Pada draf ini, hasil eksperimen belum tersedia.

1.2 Rumusan Masalah

Penelitian ini diarahkan oleh pertanyaan berikut.

1. **RQ1:** Bagaimana kesetaraan bidang konfigurasi yang ditetapkan dan kesehatan layanan, yang dilaporkan secara terpisah, berubah setelah intervensi dan prosedur koreksi pada mode imperatif terotomasi berbasis `kubectl` dibandingkan dengan mode GitOps menggunakan ArgoCD?

2. **RQ2:** Bagaimana distribusi waktu deteksi, waktu hingga tersedia bukti penyimpangan yang dapat diverifikasi, dan waktu pemulihan konfigurasi penuh berbeda antara kedua mode untuk setiap skenario yang dianalisis?

RQ1 dan RQ2 dijawab per skenario. HPA normal tetap aktif pada kedua mode; A menjadi kontrol HPA, bukan bukti efikasi ArgoCD memulihkan replika yang diabaikan. A, uji batas visibilitas F, dan kasus deskriptif G dilaporkan terpisah dari estimasi B–E. Tidak adanya OutOfSync bukan waktu deteksi nol. R3 tidak diukur karena audit API server tidak tersedia dan tidak dilaporkan sebagai nol atau tidak ada perbedaan. Penyesuaian tersebut tetap draf untuk tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan.

1.3 Batasan Masalah

Ruang lingkup penelitian dibatasi sebagai berikut.

1. Perbandingan mencakup satu prosedur imperatif terotomasi berbasis `kubectl` dan satu implementasi GitOps berbasis ArgoCD. Tidak ada intervensi manusia dalam waktu deteksi atau pemulihan yang diukur; hasil tidak mewakili kecepatan operator atau semua implementasi kedua paradigma.

2. Beban uji formal adalah satu stack InvenioRDM multi-service pada satu klaster Kubernetes di namespace `invenio`. Tidak ada stack kembar `exp-*` dan tidak ada eksekusi paralel A/B. Karena stack yang sama dipakai bergantian, non-isolasi terhadap beban latar dan dependensi bersama menjadi keterbatasan eksplisit. Generalisasi dibatasi pada konfigurasi apparatus yang dilaporkan.

3. Surrogat S1 dan S2 merupakan artefak historis/offline untuk validasi instrumen. Keduanya tidak dipakai dalam desain formal, tidak digabungkan dengan data formal, dan tidak mendukung kesimpulan tentang InvenioRDM.

4. Intervensi yang direncanakan meliputi perubahan replika (A), image (B), ConfigMap (C), penghapusan sumber daya (D), patch requests/limits (E), sumber daya pada namespace di luar cakupan aplikasi (F), dan anotasi metadata Secret (G). A adalah kontrol HPA; F adalah batas visibilitas sumber daya tak terkelola; G bukan kasus *self-heal* ArgoCD dan dianalisis deskriptif.

5. R1 menilai kesetaraan inventaris dan bidang deklaratif yang ditetapkan dengan aturan sama pada kedua mode; kesehatan dilaporkan terpisah. R2 mencakup waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi penuh. R3 tidak diukur karena audit API server tidak tersedia; R3 dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai nol atau tidak ada perbedaan. Nilai Secret tidak dipublikasikan.

6. Analisis dibatasi pada hasil per skenario. Sampel formal adalah 15 pengulangan per skenario per mode (10 R2 dan 5 R1; total 210 run); angka tersebut bukan jaminan kecukupan daya. Data tersensor, kegagalan injeksi, dan kegagalan instrumentasi dilaporkan terpisah; tidak ada nilai hilang yang diganti nol.

1.4 Tujuan Penelitian

Penelitian ini bertujuan merancang dan, setelah protokol disetujui serta apparatus divalidasi, mengevaluasi perbedaan operasional antara mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD pada beban uji formal multi-service InvenioRDM. Evaluasi mengoperasionalkan RQ1 melalui kesetaraan konfigurasi dan kesehatan yang terpisah serta RQ2 melalui waktu deteksi, bukti, dan pemulihan konfigurasi. R3 tidak diukur. Tujuan ini tidak menyatakan sebelumnya mode mana yang lebih baik.

1.5 Manfaat Penelitian

Jika protokol berhasil divalidasi dan data dikumpulkan, penelitian ini diharapkan dapat:

1. memberikan deskripsi empiris yang dibatasi pada *workload*, skenario, dan konfigurasi yang benar-benar diuji;
2. mendokumentasikan definisi pengukuran serta aturan penanganan data tersensor agar dapat ditinjau dan direplikasi; dan
3. menyediakan informasi kontekstual bagi praktisi tanpa mengubah hasil satu kasus menjadi rekomendasi universal tentang GitOps atau pengelolaan imperatif.

1.6 Sistematika Penulisan

Skripsi ini disusun sebagai berikut:

- Bab I menjelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.
- Bab II membahas konsep drift, pendekatan pengelolaan konfigurasi, studi empiris terkait, serta ruang kontribusi yang dibatasi.
- Bab III menjelaskan rancangan perbandingan, *workload*, skenario, definisi metrik, protokol, analisis, dan ancaman validitas.
- Bab IV menyediakan struktur pelaporan setup dan bukti untuk R1–R2 serta batas pengukuran R3; hasil empiris hanya dapat diisi setelah eksperimen formal tervalidasi.

- Bab V merangkum status kesimpulan, keterbatasan, serta rekomendasi yang dapat ditindaklanjuti setelah hasil tersedia.

BAB II

TINJAUAN PUSTAKA

2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah ketidakcocokan antara keadaan aktual sistem dan keadaan yang ditetapkan sebagai acuan deklaratif (Pohjola, 2025). Drift berbeda dari cacat statis: konfigurasi yang tidak aman dapat tetap cocok dengan acuannya, sedangkan perubahan aktual yang tidak cocok dengannya merupakan drift meskipun nilai aktual belum dinilai dampaknya. Penelitian ini mengukur kesetaraan terhadap deklarasi, bukan kebenaran semantik atau keamanan seluruh manifest.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola (2025) memberikan kerangka konseptual untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Penelitian tersebut mengidentifikasi tiga faktor pendorong utama penyimpangan: (1) keahlian manusia (*human expertise*) yang tidak konsisten; (2) tekanan waktu (*time pressure*) yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) ketiadaan mekanisme deteksi otomatis pada *pipeline deployment*. Meskipun penelitian tersebut tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklaratif.

2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada klaster: Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik (Pohjola, 2025).

3. Kurangnya mekanisme *rollback* yang terintegrasi: kemampuan untuk mengembalikan perubahan bergantung pada pencatatan versi, ketersediaan nilai dasar, dan fitur yang dikonfigurasi pada alat pengelolaan (Rahman et al., 2023).

4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar de facto untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

2.2.2 Git dan *Version Control*

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth*

yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-*review*, dan dikembalikan (*rollback*) secara andal.

2.2.3 *Continuous Integration* dan *Continuous Deployment*

Continuous Integration (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

2.2.4 *Infrastructure-as-Code* (IaC)

Infrastructure-as-Code (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reprodusibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* klaster.

2.2.5 Paradigma Deklaratif vs Imperatif

Dalam paradigma imperatif, tindakan pengelolaan dinyatakan sebagai perintah atau urutan tindakan yang mengubah keadaan sistem (*how to achieve the state*). Dalam paradigma deklaratif, keadaan yang diinginkan dinyatakan dan pengendali bertugas mendekatinya (*what state to achieve*) (Scott, 2015). Keduanya dapat diotomasi; perbedaan untuk penelitian ini adalah prosedur koreksi imperatif tetap yang dijalankan melalui `kubectl` dibandingkan dengan rekonsiliasi ArgoCD.

2.3 Manajemen Konfigurasi Infrastruktur

Infrastructure state management mengatur bagaimana keadaan infrastruktur didefinisikan, diterapkan, dan dipertahankan. Pendekatan deklaratif menyimpan keadaan yang diinginkan dalam konfigurasi, sedangkan pendekatan imperatif menyatakan tindakan untuk mengubah keadaan (Morris, 2020; Limoncelli et al., 2018). Otomasi, pemeriksaan berulang, dan pencatatan dapat tersedia pada kedua pendekatan; karakteristik tersebut perlu ditetapkan pada tingkat implementasi yang dibandingkan.

2.4 *Continuous Deployment* dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

GitOps mendukung pencatatan versi deklarasi melalui Git, peninjauan perubahan, dan penggunaan revisi sebelumnya sebagai acuan pemulihan (Yuen et al., 2021). Riwayat Git tidak otomatis merekam perubahan langsung pada klaster; tindakan tersebut memerlukan bukti dari pengendali atau audit API. Pada model *pull-based*, agen di dalam klaster dapat menarik deklarasi tanpa pipeline eksternal menerapkan setiap perubahan ke API klaster, tetapi kebutuhan kredensial dan akses operasional tetap bergantung pada rancangan serta konfigurasi sistem (Utami and Fatta, 2021). Klaim adopsi dan manfaat GitOps juga muncul dalam artikel industri CNCF 2025; artikel tersebut dipakai hanya sebagai konteks industri, bukan sebagai bukti *peer-review* (Lopez and Narang, 2025).

2.4.1 Performa *Deployment* dan *Anti-Pattern*

Forsgren dkk. (Forsgren et al., 2018) membahas *deployment frequency*, *lead time for changes*, *time to restore service*, dan *change failure rate* sebagai metrik performa pengembangan dan operasi. Metrik organisasi tersebut tidak identik dengan pengukuran eksperimen ini dan tidak dengan sendirinya membuktikan bahwa satu prosedur selalu lebih unggul. Penelitian ini menggunakan konstruk yang lebih terbatas: kesetaraan bidang konfigurasi, waktu deteksi dan pemulihan konfigurasi, serta bukti keterlacakan.

ArgoCD menyediakan pengendali aplikasi yang membandingkan sumber daya teramati dengan deklarasi repositori dan, jika kebijakan serta cakupannya mengizinkan, dapat melakukan sinkronisasi atau koreksi otomatis (Argo Project, 2024). Perilaku aktual bergantung pada konfigurasi aplikasi, izin, ketersediaan pengendali, jenis sumber daya, dan cakupan inventaris. Oleh sebab itu, rekonsiliasi tidak menjamin setiap perubahan akan selalu ditemukan atau dipulihkan.

2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.5.1 *Convergence* Menuju *Desired State*

Pengendali deklaratif secara berkala membandingkan keadaan aktual dengan keadaan yang diinginkan dan dapat melakukan tindakan korektif sesuai kebijakan yang berlaku (Argo Project, 2024). Keberhasilan konvergensi bukan sifat yang dapat diasumsikan untuk semua sumber daya atau gangguan: hasil dipengaruhi oleh cakupan aplikasi, izin, ketersediaan komponen, konfigurasi sinkronisasi, dan sifat sumber daya.

Perintah imperatif tidak dengan sendirinya menyediakan rekonsiliasi berkelanjutan. Namun, prosedur imperatif dapat diotomasi dengan pemeriksaan berkala dan tindakan koreksi yang telah ditentukan. Karena penelitian ini membandingkan prosedur `kubectl` terotomasi dengan ArgoCD, bukan respons manusia, perbedaan waktu tidak ditafsirkan sebagai perbedaan kecepatan operator.

2.5.2 *Audit Trail* sebagai *Social Control Mechanism*

Riwayat Git dapat menyediakan metadata dan perubahan deklarasi, sedangkan log pengendali dan audit API dapat memberi bukti tambahan tentang penerapan aktual (CNCF GitOps Working Group, 2021). Riwayat tersebut tidak otomatis membuktikan siapa yang melakukan perubahan pada klaster atau apakah semua tindakan terekam; kelengkapan bergantung pada tata kelola repositori, identitas, konfigurasi audit, dan pengumpulan bukti. Namun, R3 tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia; keterlacakan dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai hasil.

2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan

Runbook adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem. Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Dalam rancangan penelitian ini, prosedur imperatif bukan runbook yang dipandu kepada operator. Mode imperatif menggunakan pemeriksa dan korektor otomatis yang tetap, dengan pembandingan deklarasi dan keadaan aktual serta tindakan `kubectl` yang telah ditetapkan. Mode ini diperoleh dengan menjeda *auto-sync*/*self-heal* ArgoCD pada stack yang sama, lalu memulihkannya. Pilot perlu memverifikasi bahwa prosedur tersebut bekerja tanpa menerima identitas skenario atau label injeksi dan menyimpan bukti yang sama jenisnya dengan evaluator bersama.

2.7 Paradigma *Deployment* Alternatif

Pilihan pengelolaan konfigurasi mencakup rekonsiliasi berbasis tarik, pipeline dorong, alat konfigurasi yang dijalankan sesuai kebutuhan, serta otomasi imperatif khusus. Perbedaan autentikasi, cakupan sumber daya, pencatatan, dan frekuensi eksekusi bergantung pada konfigurasi implementasi; tidak diasumsikan bahwa seluruh pipeline dorong mengekspos kredensial atau bahwa semua alat imperatif tidak memiliki audit. Studi Shrestha dan Ali secara khusus membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift (Shrestha and Ali, 2024).

Penelitian ini membatasi pembanding pada satu prosedur imperatif terotomasi berbasis `kubectl` dan satu instalasi ArgoCD. Kontras ini tidak merupakan batas atas efek yang berlaku bagi semua alat atau konteks; temuan nantinya hanya berlaku bagi implementasi dan workload yang benar-benar diuji.

2.8 Evaluasi Empiris GitOps dan ArgoCD

Studi terkait telah menguji sejumlah kombinasi alat dan tujuan penelitian. Shrestha dan Ali (Shrestha and Ali, 2024) membandingkan GitOps dengan Ansible pada waktu induksi, penemuan, dan pemulihan drift. Kagganti Nataraja (Nataraja,

2025) mendeskripsikan perbandingan terkontrol pada klaster Minikube satu node dengan konfigurasi keamanan yang berbeda. Matubber (Matubber, 2025) mengevaluasi pengelolaan infrastruktur Kubernetes berbasis GitOps dan mencakup pembanding lama yang semi-terotomasi. Ketiganya menunjukkan bahwa penelitian komparatif atau evaluasi berbasis waktu telah ada; abstrak yang tersedia tidak cukup untuk menyimpulkan ada atau tidaknya uji statistik tertentu.

Utami dan Al Fatta (Utami and Fatta, 2021) membahas model deployment deklaratif berbasis tarik dibandingkan dengan model berbasis dorong. Paavola (Paavola, 2021) membahas pengelolaan beberapa aplikasi Kubernetes menggunakan prinsip GitOps, sedangkan D'Amore (D'Amore, 2021) mendeskripsikan implementasi ArgoCD pada lingkungan Kubernetes hybrid cloud. Studi tersebut memberi konteks implementasi, tetapi tidak diperlakukan di sini sebagai bukti bahwa salah satu prosedur selalu lebih unggul.

Perbandingan penelitian perlu dibaca sesuai tujuan, alat, lingkungan, konfigurasi, dan hasil yang benar-benar dilaporkan oleh masing-masing sumber. Draf ini tidak menyatakan bahwa penelitian terdahulu tidak terkontrol, tidak menggunakan statistik, atau tidak memiliki baseline apabila pernyataan itu tidak dapat dipastikan dari abstrak yang tersedia.

2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Penelitian ini merupakan studi kasus komparatif terkontrol: satu kasus diamati pada kondisi dasar, satu intervensi, dan kondisi pascaintervensi, dengan dua mode pengelolaan yang dijalankan berurutan (Runeson and Höst, 2009). Limoncelli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Rancangan penelitian ini menetapkan mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD sebagai dua perlakuan pada satu stack. Kesetaraan *workload*, nilai dasar, inventaris pengukuran, dan prosedur injeksi direncanakan untuk mengurangi perbedaan yang tidak relevan. Karena kedua implementasi dapat berbeda dalam lebih dari satu komponen teknis dan tidak ada isolasi penuh, hasil tidak langsung diatribusikan sebagai efek universal paradigma; interpretasinya

dibatasi pada prosedur dan kasus yang benar-benar diuji (Wohlin et al., 2012).

Analisis yang direncanakan bersifat per skenario dan eksploratif. Ringkasan distribusi, jumlah observasi lengkap dan tersensor, serta ukuran efek dan interval kepercayaan dilaporkan bersama-sama; uji komparatif hanya digunakan jika asumsi dan struktur data mendukungnya. Sampel formal 15 pengulangan per skenario per mode (210 run) tidak dianggap sebagai bukti daya statistik yang memadai (Dybå et al., 2006; Kitchenham and Madeyski, 2024). Rincian definisi operasional dan batas interpretasi terdapat pada Bab III.

2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada klaster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.

2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, TLS, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Pengujian formal dalam rancangan ini menggunakan beban uji komposit multi-service InvenioRDM pada satu stack produksi-like. Inventaris dan komponen aktual harus dicatat dari konfigurasi yang divalidasi sebelum eksperimen; jumlah komponen yang pernah disebut sebagai target belum diverifikasi. Beban uji merupakan konteks pengukuran, bukan dasar untuk menggeneralisasi hasil ke seluruh *workload* multi-service. Surrogat S1/S2 historis/offline tidak dipakai dalam desain formal dan tidak memperluas sampel formal.

2.11 Ruang Kontribusi Penelitian

Studi komparatif mengenai GitOps dan pengelolaan konfigurasi telah ada, termasuk evaluasi terhadap Ansible, pengujian terkontrol pada Minikube, dan pengukuran yang mencakup *baseline* semi-terotomasi. Karena itu, penelitian ini tidak berangkat dari klaim bahwa perbandingan terdahulu tidak ada atau bahwa keunggulan GitOps telah terbukti secara universal.

Kontribusi yang diklaim bersifat terbatas dan dapat diperiksa: (1) perbandingan empiris pada *deployment* InvenioRDM multi-service nyata, bukan *workload* sintetis; (2) dua mode yang dijalankan berurutan pada satu stack produksi-like, dengan mode imperatif didefinisikan sebagai penjedaan ArgoCD ditambah pemeriksa/*runbook* `kubectl`, sehingga perbandingan tidak memerlukan stack kembar; (3) pengukuran R1/R2 yang dapat direproduksi dengan kepemilikan HPA, batas visibilitas F, dan kasus G yang dipisahkan dari efikasi; serta (4) bukti batas bahwa hasilnya berlaku untuk kasus dan apparatus yang diuji, bukan klaim superioritas GitOps universal. R3 tidak diukur karena audit API server tidak tersedia dan dibahas sebagai keterbatasan serta pekerjaan lanjutan.

Hasil yang kelak diperoleh hanya dapat menjawab pertanyaan pada skenario, *workload*, konfigurasi, dan resolusi instrumentasi yang benar-benar diuji. Besaran maupun arah perbedaan belum dapat dipastikan sebelum pengumpulan serta analisis data formal.

BAB III

METODE PENELITIAN

3.1 Deskripsi Umum dan Status Protokol

Penelitian ini dirancang sebagai studi kasus komparatif terkontrol (*controlled single-case comparative study*) atas dua mode pengelolaan pada satu stack InvenioRDM produksi-like: mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD. Pertanyaan yang dituju ialah bagaimana kedua mode berbeda pada konsistensi konfigurasi serta waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi untuk skenario yang ditetapkan. Estimand dibatasi pada selisih hasil antarmode dalam konteks beban uji InvenioRDM dan apparatus eksperimen yang diverifikasi; estimand ini bukan efek universal seluruh paradigma imperatif atau GitOps.

Kedua mode bukan dua *deployment* terpisah. Pada mode imperatif, *auto-sync* dan *self-heal* ArgoCD dijeda, sedangkan pemeriksa dan *runbook* otomatis tetap bekerja dengan `kubectl`; setelah blok selesai, ArgoCD dipulihkan dan diverifikasi. Pada mode GitOps, ArgoCD aktif. Tidak ada eksekusi paralel A/B, tidak ada stack kembar `exp-*`, dan tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran. Karena satu stack dipakai bergantian, perlakuan tidak terisolasi dari beban latar, dependensi bersama, atau keadaan klaster yang berubah; non-isolasi tersebut merupakan keterbatasan eksplisit.

Naskah ini memuat rancangan protokol, bukan laporan pelaksanaan. Protokol masih memerlukan tinjauan pembimbing, dan apparatus eksperimen yang tersedia belum divalidasi untuk mengimplementasikan seluruh definisi operasional di bawah. Versi perangkat lunak, kesiapan klaster, inventaris sumber daya formal, dan data eksperimen belum ditetapkan atau diverifikasi dalam naskah ini. Pilot terbatas pada stack yang sama bukan data formal. Karena itu, uraian rencana tidak boleh dibaca sebagai bukti bahwa eksperimen telah dilaksanakan.

Penyesuaian kepemilikan pengendali tetap berstatus draf untuk tinjauan pemilik penelitian dan pembimbing: HPA normal aktif pada kedua mode dan skenario A menjadi kontrol respons HPA. A tidak digabungkan ke efikasi ArgoCD pada B–E; F adalah kasus batas visibilitas tersendiri; G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. Seluruh eksekusi formal dinonaktifkan. Instrumen saat ini memvalida-

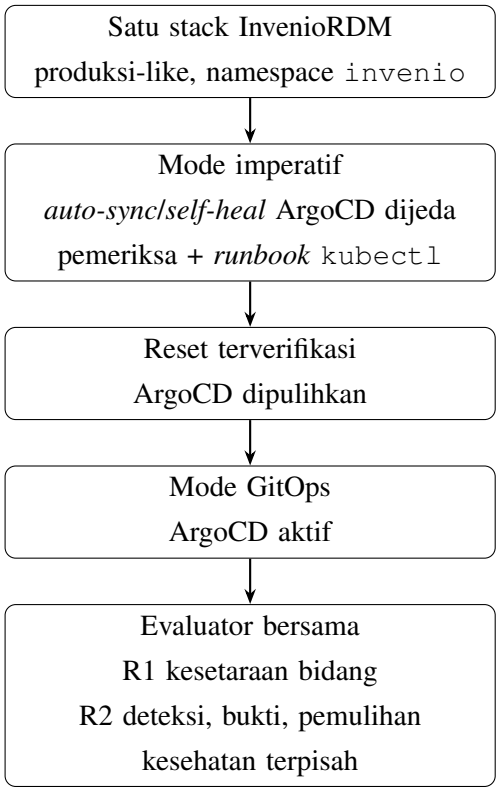
si bukti tersimpan secara offline, bukan menjalankan injeksi, *deployment*, atau reset langsung; adapter polling, injeksi, pemulihan, dan SMTP belum selesai. Audit API server tidak tersedia; ini tidak menghambat *deployment*, tetapi menutup R3.

3.2 Rancangan Perbandingan

Pengendalian kondisi awal, pengulangan, pencatatan penyimpangan protokol, dan pembatasan inferensi mengikuti prinsip studi kasus dan evaluasi empiris perangkat lunak (Wohlin et al., 2012; Runeson and Höst, 2009). Karena kasus tidak dipilih acak dari populasi dan kedua mode berurutan pada stack yang sama, rancangan tidak dianggap sebagai penugasan acak klaster dari suatu populasi (Cook and Campbell, 1979).

Unit analisis adalah satu injeksi penyimpangan yang diterima pada satu mode dalam satu pengulangan. Mode imperatif menggunakan otomasi tetap: pemeriksa berkala membandingkan keadaan aktual dengan inventaris dan nilai yang diinginkan, menyimpan perbedaan yang teramati, lalu menjalankan prosedur koreksi `kubectl` yang telah ditentukan. Kepemilikan rekonsiliasi ArgoCD dijeda secara terkendali dengan menonaktifkan *auto-sync*/*self-heal* pada `apps` dan `invenio-bootstrap`, lalu dipulihkan pada blok `finally` dan diverifikasi (Argo Project, 2025). Mode GitOps menggunakan ArgoCD dengan rekonsiliasi otomatis terhadap deklarasi Git. Tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran; waktu respons manusia maupun tingkat keahlian operator bukan variabel penelitian.

Kedua mode memakai beban uji, nilai dasar, definisi bidang yang diperiksa, serta prosedur injeksi dan pengukuran yang setara. Injeksi dilakukan satu per satu, bukan serentak, dan urutan mode serta skenario diacak dalam blok pengujian setelah kondisi dasar dipulihkan. Pengendali koreksi hanya menerima keadaan aktual dan deklarasi yang diperlukan untuk membandingkannya; identitas skenario tidak diteruskan sebagai label, termasuk pada skenario Secret. Rancangan ini membandingkan dua implementasi operasional yang telah ditentukan, bukan mengisolasi seluruh faktor yang mungkin membedakan setiap produk atau organisasi.

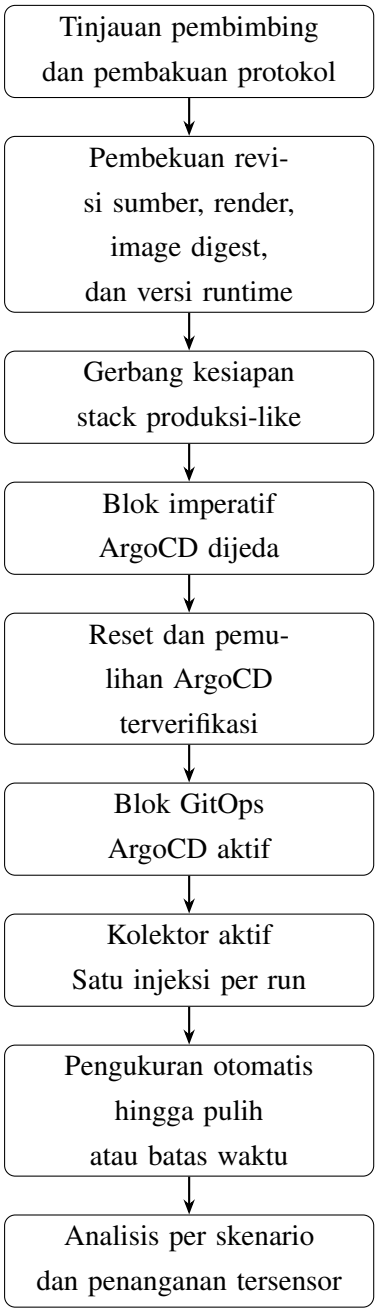


Gambar 3.1: Rancangan dua mode pengelolaan berurutan pada satu stack InvenioRDM; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah.

Gambar 3.1 menunjukkan bahwa injeksi, mekanisme pengelolaan, dan evaluator merupakan bagian yang dibedakan secara eksplisit. Hasil hanya akan ditafsirkan untuk konfigurasi apparatus yang benar-benar dicatat saat eksperimen; HPA adalah pengendali bersama pada kedua mode.

3.3 Alur Penelitian

Tahapan yang direncanakan meliputi peninjauan protokol, pembekuan sumber dan versi, validasi instrumen offline, pengujian formal, dan analisis per skenario. Surrogat S1/S2 merupakan artefak historis/offline; keduanya tidak dipakai dalam desain formal dan tidak masuk ke sampel formal.



Gambar 3.2: Bagan alir protokol dengan pembekuan sumber sebelum blok pengukuran dan tanpa surrogat pada desain formal.

Gambar 3.2 menempatkan pembekuan sumber dan validasi sebelum pengumpulan data formal serta mensyaratkan pemulihan kondisi dasar di antara pengulangan. Tidak ada jeda buatan setelah injeksi untuk menunggu respons manusia.

3.4 Beban Uji dan Lingkungan

Pengujian formal menggunakan beban uji komposit multi-service InvenioRDM dengan sasaran sedikitnya 16 komponen aplikasi dan infrastruktur pendukung sesuai ruang lingkup proposal. Sasaran tersebut bukan jumlah komponen yang telah diverifikasi. Inventaris formal mencakup seluruh sumber daya dalam manifest yang disetujui; daftar komponen, batas pengelolaan, dan versinya dicatat dari artefak yang benar-benar digunakan. S1/S2 merupakan surrogat historis/offline, bukan pengganti beban formal, dan tidak boleh digabungkan ke hasil atau analisis formal.

Rancangan membatasi pengujian pada satu klaster Kubernetes, satu stack InvenioRDM produksi-like di namespace `invenio`, dan satu implementasi GitOps, yaitu ArgoCD. Tidak ada namespace kembar `exp-*`, tidak ada *deployment* kembar, dan tidak ada eksekusi paralel. Spesifikasi serta versi aktual klaster, node, komponen beban uji, dan ArgoCD harus diverifikasi dan dicatat sebelum pengujian; nilai yang pernah disebut sebagai sasaran pada draf sebelumnya belum menjadi fakta eksperimen. Karena satu stack dipakai bergantian, komponen infrastruktur bersama tidak dapat diisolasi dari perlakuan; injeksi dan koreksi dibatasi pada objek dalam kontrak bidang.

Konteks *deployment* yang dilaporkan pemilik adalah Kubernetes 1.32.6/RKE2, containerd, dan Ubuntu 24.04. Penunjuk sumber otoritatif ada pada `experiments/deployment-source.json`, dengan revisi referensi `6cacea6642a7c1690243d3b54130fa08216a9595`. Application `argocd/apps/invenio-bootstrap.yaml` memilih `k8s/apps/invenio/kustomization.yaml`; konfigurasi sumber rekonsiliasi menetapkan 30 detik, bukan target lama 60 detik. Revisi referensi bukan bukti revisi yang diterapkan. Sebelum formal, rekam revisi runtime aktual, render dan digest-nya, image digest, inventaris, serta interval pengendali; tidak dibuat sumber manifest editable kedua di repositori penelitian.

3.4.1 Gerbang Kesiapan

Pengulangan formal hanya dimulai setelah seluruh prasyarat berikut diperiksa dan dicatat: (1) manifest dan inventaris sumber daya formal telah dibekukan; (2) stack berada pada kesetaraan nilai yang ditetapkan dan kesehatan layanan telah memenuhi kriteria kesiapan; (3) *auto-sync*, *self-heal*, cakupan aplikasi, dan kebijakan prune ArgoCD telah diverifikasi, termasuk prosedur penjedaan dan pemulihan untuk mode

imperatif; (4) pemeriksa imperatif, pengumpul bukti, serta pembanding bidang telah lulus validasi offline; (5) jam pencatat waktu tersinkronisasi; dan (6) tidak ada pengujian atau perubahan lain yang berlangsung pada lingkup sumber daya yang diamati. Ambang kesehatan, batas beban, interval ArgoCD, serta batas waktu pengamatan dibakukan setelah pilot dan tinjauan pembimbing; naskah ini tidak menyatakan nilai aktualnya.

Jika prasyarat gagal, injeksi tidak dimulai. Gangguan setelah injeksi dicatat sebagai kejadian pengujian, bukan dihapus tanpa jejak. Kriteria penghentian, nilai batas waktu, dan aturan pengulangan harus ditetapkan sebelum data formal dikumpulkan.

Gerbang CPU dan memori tetap ketat di bawah 70% pada setiap node; memori memakai *working set* metrics-server dibagi *allocatable*. Bukti metrik, inventaris, resource requests/limits, pod Running dan Ready, serta reset aplikasi wajib lengkap. Audit API tidak tersedia; ini tidak menjadi gerbang *deployment*, tetapi menutup R3. Kondisi worker02 sekitar 71% yang dilaporkan pemilik, SMTP placeholder, kapasitas satu stack, serta *private-draft seed*/recovery yang tertunda tidak dianggap teratasi. Bootstrap/migrasi/fixtures/indexing/queue tidak boleh dipicu sebagai bagian repair rutin tanpa desain lifecycle yang ditinjau. Interval rekonsiliasi repositori, *watch* kejadian, *retry self-heal*, dan evaluator dibedakan.

3.5 Intervensi dan Skenario

Intervensi adalah satu perubahan terkontrol terhadap keadaan sumber daya dalam satu pengulangan. Skenario A–E menyasar sumber daya yang termasuk inventaris aplikasi; F menguji batas visibilitas untuk sumber daya di luar cakupan aplikasi; G adalah kasus deskriptif pada metadata Secret dan bukan *self-heal* ArgoCD.

Tabel 3.1: Skenario intervensi yang direncanakan

Skenario	Intervensi	Batas interpretasi dan analisis
A	Mengubah jumlah replika Deployment yang dikendalikan HPA	Kontrol HPA normal pada kedua mode; `/spec/replicas` dikecualikan dari R1 statis; bukan koreksi ArgoCD
B	Mengubah referensi image Deployment terkelola	Drift terkelola; perubahan bidang yang telah ditetapkan
C	Mengubah bidang ConfigMap terkelola	Drift terkelola; tidak mengubah deklarasi sumber kebenaran
D	Menghapus sumber daya terkelola	Drift terkelola; pemulihan dinilai terhadap inventaris dan nilai dasar
E	Mengubah bidang sumber daya terkelola melalui patch	Drift terkelola; hanya bidang yang telah ditetapkan yang dinilai
F	Membuat sumber daya pada namespace di luar cakupan aplikasi	Batas visibilitas sumber daya tak terkelola; deskriptif, bukan keberhasilan atau kegagalan *self-heal*
G	Mengubah anotasi metadata Secret terkelola	Bukan kasus *self-heal* ArgoCD; pemulihan oleh harness; deskriptif dan tidak dipool

Pada skenario A, `/spec/replicas` web/worker dimiliki HPA dan dikecualikan dari kesetaraan statis. Respons diukur terhadap keputusan HPA yang teramati, bukan angka tetap tiga replika. Tidak adanya OutOfSync tidak berarti deteksi nol; HPA dapat memulihkan replika tanpa sinyal ArgoCD. Hasil A dilaporkan sebagai kontrol HPA terpisah. Penghormatan kepemilikan replika saat *sync* harus diverifikasi, bukan diasumsikan dari *ignoreDifferences* saja.

Pada skenario F, sumber daya di luar inventaris aplikasi tidak dianggap sebagai drift terkelola yang wajib dipulihkan ArgoCD. Pengamatan dibatasi pada apakah batas cakupan dan pencatatan inventaris membuat keberadaan sumber daya tersebut terlihat. A, F, dan G tidak digabungkan ke estimasi utama B–E.

Pada skenario G, perubahan berupa anotasi metadata pada SealedSecret sinte-

tis; pemulihan anotasi dilakukan oleh harness, bukan oleh rekonsiliasi ArgoCD. Nilai `spec.encryptedData` tidak dibaca atau ditulis. G dianalisis secara deskriptif dan tidak dipool ke estimasi efikasi B–E. Pembekuan ImageUpdater dan pengendali SealedSecret memerlukan verifikasi sebelum formal.

3.6 Definisi Variabel dan Metrik

3.6.1 R1: Kesetaraan Konfigurasi dan Kesehatan

Kesetaraan konfigurasi dinilai menggunakan aturan yang sama pada kedua mode. Sebelum eksperimen, skema bidang yang diperiksa dan inventaris sumber daya yang diharapkan dibekukan. Bidang dinamis yang tidak relevan bagi deklarasi dikecualikan secara eksplisit. Untuk setiap sumber daya yang diharapkan, sumber daya dinyatakan cocok jika keberadaannya sesuai inventaris dan semua bidang terpilih sama dengan nilai yang diinginkan.

$$R1_{eq} = \frac{\text{jumlah sumber daya terkelola yang cocok}}{\text{jumlah sumber daya dalam inventaris yang diharapkan}} \times 100\%.$$

Evaluator menghitung $R1_{eq}$ dari pembacaan keadaan aktual dan inventaris yang sama bentuknya untuk kedua mode; status *Synced* ArgoCD bukan pengganti aturan kesetaraan tersebut. Nilai dicatat pada kondisi dasar, pengamatan pertama setelah injeksi, dan setelah prosedur koreksi. Jika koreksi mendahului pengamatan pascainjeksi, kondisi sesaat tersebut ditandai tidak teramati; kondisi drift tidak direkonstruksi seolah-olah telah diukur. Nilai Secret dibandingkan secara aman di memori; nilai mentah maupun hash yang dapat digunakan untuk menebak isinya tidak ditulis ke log, gambar, atau laporan.

Kontrak bidang mencakup image, requests/limits, ConfigMap, Secret dan bidang Service yang relevan. Bidang dinamis HPA tidak dibandingkan dengan nilai replika statis Git. Bidang yang tidak diperiksa atau kegagalan API menggagalkan bukti; sumber daya yang benar-benar tidak ada pada daftar lengkap memberi ketidakcocokan. Kesehatan *rollout* tidak menggantikan pemeriksaan bidang tersebut.

Kesehatan layanan dilaporkan sebagai hasil terpisah berdasarkan kriteria kesiapan yang ditetapkan sebelum eksperimen, misalnya kondisi kesiapan Pod dan status layanan yang memang tersedia pada *workload*. Kesehatan tidak dimasukkan ke pembilang kesetaraan konfigurasi. Sumber daya ekstra di luar inventaris, termasuk F,

dilaporkan terpisah dan tidak mengubah penyebut R1.

3.6.2 R2: Deteksi, Ketersediaan Bukti, dan Pemulihan

Semua waktu berasal dari satu kolektor pada host pencatat. Setiap kejadian diberi waktu jam monotonik dalam detik dengan pecahan desimal serta cap waktu UTC untuk audit; durasi dihitung dari jam monotonik, bukan dari selisih jam dinding. Pengamatan diaktifkan sebelum injeksi dan berlangsung pada interval nominal satu detik. Cap waktu adalah waktu pengamatan pertama, bukan waktu kejadian yang diinterpolasi; interval aktual dan jeda polling dicatat. Interval ini merupakan resolusi pengamatan yang direncanakan dan harus divalidasi pada pilot. Interval pemeriksa imperatif dan jadwal rekonsiliasi ArgoCD dicatat tersendiri dari interval evaluator, dibekukan sebelum pengujian formal, dan tidak disesuaikan setelah hasil terlihat.

- t_0: waktu kolektor mulai mengirim permintaan injeksi ke Kubernetes API. Waktu penerimaan respons API juga dicatat sebagai t_a, sehingga $t_a - t_0$ mendokumentasikan latensi permintaan. Injeksi diterima hanya jika operasi API berhasil dan objek respons atau pembacaan balik memverifikasi perubahan bidang sasaran. Durasi sejak t_0 mencakup latensi permintaan, bukan semata-mata waktu respons setelah perubahan tersimpan. Injeksi gagal atau tidak terverifikasi dicatat sebagai kegagalan injeksi, bukan sebagai waktu deteksi nol.

- t_d: waktu pertama mekanisme pengelolaan memberi sinyal deteksi berdasarkan perbedaan dari nilai yang diinginkan. Pada mode imperatif, sinyal berasal dari pemeriksa imperatif; pada mode GitOps, dari status perbedaan ArgoCD yang relevan untuk B–E. Tidak ada sinyal skenario tersembunyi yang diberikan kepada mekanisme tersebut. Sinyal HPA dicatat sebagai sinyal HPA, bukan deteksi ArgoCD.

- t_e: waktu pertama bukti dari mekanisme pengelolaan tersimpan dan mengidentifikasi sumber daya serta bidang yang berbeda terhadap acuan. Bukti harus dapat diperiksa ulang tanpa memakai label injeksi yang tersembunyi; untuk Secret cukup dicatat identitas bidang dan hasil ketidakcocokan, bukan isinya. Kriteria ini hanya mengukur bukti deviasi, bukan identifikasi penyebab. t_e dapat sama dengan t_d, tetapi keduanya tetap dicatat terpisah. Pengumpulan bukti tidak menunda koreksi pada salah satu mode.

- t_r: waktu pengamatan pertama ketika inventaris dan seluruh bidang R1 kembali cocok dengan nilai yang diinginkan, dengan kecocokan dikonfirmasi pada pembacaan berikutnya. Waktu yang dilaporkan tetap waktu pengamatan pertama; konfirmasi hanya mencegah keadaan sesaat dinyatakan pulih. Kesehatan layanan dilaporkan terpisah.

Durasi utama adalah waktu deteksi $t_d - t_0$, waktu ketersediaan bukti $t_e - t_0$, dan waktu pemulihan konfigurasi penuh $t_r - t_0$. Selang dari deteksi ke bukti ($t_e - t_d$) dapat ditampilkan sebagai rincian tambahan. t_r berarti pemulihan kesetaraan inventaris dan bidang yang diukur, bukan klaim bahwa seluruh layanan sehat atau tidak mengalami dampak lain.

Sinyal deteksi mode imperatif berasal dari pemeriksaan otomatis terhadap keadaan aktual; setelah sinyal muncul, prosedur koreksi tetap menerapkan deklarasi yang diinginkan menggunakan tindakan `kubectl` yang telah ditetapkan. Mode GitOps menggunakan rekonsiliasi ArgoCD. Kedua prosedur berjalan otomatis tanpa waktu reaksi manusia dan tanpa jeda buatan. Keputusan koreksi menggunakan hasil perbandingan umum, bukan label A–G yang disembunyikan dari responden.

3.6.3 R3: Batas Pengukuran

R3 (bukti keterlacakan) tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi, sehingga klaim keterlacakan tidak dapat dibuat untuk kedua mode. Ketiadaan audit tidak menghambat *deployment* atau pengukuran R1/R2; ia hanya menutup R3. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik. Unsur pelaku, objek dan bidang, waktu dan urutan, serta tindakan dan hasil didokumentasikan sebagai keterbatasan dan pekerjaan lanjutan setelah audit diaktifkan.

3.7 Protokol Pengulangan dan Penanganan Kegagalan

Satu pengulangan mengikuti urutan berikut:

1. Periksa gerbang kesiapan dan simpan bukti kondisi dasar, inventaris, kesehatan, serta status instrumentasi. Bekukan revisi sumber, SHA-256 render, image digest, dan versi runtime.

2. Aktifkan koleksi sinyal deteksi, bukti, kesetaraan, dan kesehatan sebelum mengirim injeksi; pastikan mekanisme pengelolaan sesuai mode telah aktif. Untuk mode imperatif, jeda *auto-sync*/*self-heal* pada `apps` lalu `invenio-bootstrap`, simpan spesifikasi asli, dan verifikasi penjedaan. Untuk mode GitOps, verifikasi keduanya aktif. Jalankan hanya satu skenario pada satu mode. Catat t_0, respons API pada t_a, dan verifikasi penerimaan injeksi tanpa menambahkan label atau identitas skenario ke sumber daya yang diamati.

3. Pertahankan koleksi berkala satu detik tanpa menyisipkan waktu tunggu agar manusia bereaksi atau agar drift bertahan selama durasi tertentu. Jika perubahan diterima tetapi sinyal singkat tidak teramati, laporkan keterbatasan observasi tanpa menganggap deteksi terjadi pada waktu nol.

4. Lanjutkan sampai pemulihan terkonfirmasi atau batas waktu pengamatan yang telah dibakukan tercapai. Simpan semua kejadian, jeda polling, kesalahan API, serta status penghentian.

5. Pulihkan kondisi dasar, verifikasi seluruh bidang/inventaris, kesehatan, HPA, tidak adanya objek karantina, adopsi runtime, serta keadaan aplikasi/data/queue. Untuk mode imperatif, pulihkan `invenio-bootstrap` lalu `apps` ke *automated sync*/*self-heal* asli dan verifikasi; kegagalan pemulihan dilaporkan sebagai `reset_failure`. Baru mulai pengulangan berikutnya. Kegagalan tindakan koreksi diteruskan, bukan ditelan. Tidak ada izin purge, restore, atau modifikasi data dari protokol ini.

Batas waktu maksimum pengamatan harus ditentukan dan dibekukan setelah pilot serta tinjauan pembimbing, sebelum pengumpulan formal. Tidak ada nilai batas waktu yang diklaim telah diuji dalam draf ini. Jika sinyal deteksi, bukti, atau pemulihan belum terjadi saat batas itu tercapai, hasil masing-masing ditandai tersensor kanan pada batas tersebut; waktu hilang tidak diganti nol dan pengulangan tidak dikeluarkan hanya karena pemulihannya gagal. Kegagalan injeksi, kegagalan instrumentasi, gangguan infrastruktur, dan tidak terjadinya deteksi merupakan kategori berbeda. Setiap kategori, jumlah pengulangan terdampak, alasan, serta aturan pengulangan yang telah disetujui dilaporkan.

Run dengan respons injeksi sukses tetapi tanpa bukti bahwa perubahan bidang sasaran pernah diterima tidak menjadi hasil nol; run ditandai sebagai injeksi tidak

valid dan dicatat. Jika perubahan diterima tetapi mekanisme tidak mendeteksinya sampai batas pengamatan, itu merupakan hasil deteksi tersensor, bukan kegagalan instrumen. Jika kolektor atau sumber data gagal sehingga status tidak dapat ditentukan, run ditandai kegagalan instrumentasi dan tidak ditafsirkan sebagai deteksi terlewat. Skenario F tetap diklasifikasikan sebagai pengamatan batas visibilitas dan tidak dinilai dengan asumsi pemulihan otomatis.

3.8 Pengulangan dan Analisis Data

Sampel formal adalah 15 pengulangan per skenario per mode: 10 pengulangan protokol R2 dan 5 pengulangan protokol R1, sehingga totalnya $7 \times 2 \times 15 = 210$ run pada satu kasus. Pembacaan R1 tetap disimpan pada setiap run sebagai bukti status, sedangkan 5 pengulangan R1 diarahkan pada protokol konsistensi. F dan G tetap menjalani protokol yang sama, tetapi hasil waktunya dilaporkan deskriptif dan tidak dipool ke estimasi efikasi. Sampling berlangsung pada interval nominal satu detik. Angka tersebut merupakan target yang direncanakan, bukan hasil analisis daya dan bukan jaminan kecukupan daya untuk setiap skenario (Dybå et al., 2006; Kitchenham and Madeyski, 2024). S1/S2 tidak dihitung dalam target ini. A sebagai kontrol HPA, F sebagai batas visibilitas, dan G sebagai kasus deskriptif tidak dimasukkan ke efikasi drift terkelola B–E.

Hasil disajikan per skenario dan per mode: jumlah run direncanakan, jumlah injeksi valid, jumlah kejadian teramati dan tersensor, ringkasan distribusi waktu, proporsi pemulihan pada batas pengamatan, R1, serta kesehatan terpisah. Nilai mentah yang hilang tidak diimputasi sebagai nol. Observasi yang tersensor tidak diam-diam dibuang agar hanya run yang berhasil pulih yang dibandingkan. Jika penyensoran terjadi, ringkasan dan uji yang tidak memperhitungkan penyensoran tidak ditafsirkan sebagai perbandingan lengkap; analisis komparatif untuk hasil tersebut harus ditetapkan sebelum melihat hasil, atau dilaporkan secara deskriptif tanpa klaim inferensial.

Karena mode berurutan pada stack yang sama, observasi tidak diperlakukan independen secara naif. Untuk hasil waktu yang teramati lengkap dan dapat dipasangkan berdasarkan blok skenario-pengulangan, uji nonparametrik berpasangan (misalnya Wilcoxon *signed-rank*) dapat dipertimbangkan; bila pemasangan tidak dapat dipertahankan, *Mann–Whitney U* dua sisi dilaporkan sebagai eksploratif. Ukuran efek (Cliff's δ atau Hodges–Lehmann) dilaporkan bersama interval kepercayaan bootstrap. Waktu deteksi dan pemulihan yang tersensor kanan dianalisis dengan Kaplan–

Meier serta median/*restricted mean survival*; nilai tersensor tidak diimputasi nol. Jika nilai p digunakan, ambang nominalnya adalah $\alpha = 0{,}05$ dengan penyesuaian Holm untuk keluarga perbandingan waktu yang dibakukan sebelum pengumpulan formal. Definisi keluarga uji dan penanganan penyensoran harus ditinjau bersama pembimbing sebelum protokol dibekukan; tidak ada klaim konfirmatoris dari draf ini. Penggabungan lintas skenario tidak menjadi analisis utama karena bentuk dan dampak intervensinya berbeda. R1, kesehatan, dan kasus F/G dilaporkan deskriptif; tidak ada hasil signifikansi atau besar efek yang dinyatakan sebelum data dikumpulkan.

3.9 Ancaman terhadap Validitas dan Keterbatasan Rancangan

3.9.1 Validitas Konstruk

Validitas konstruk dibatasi oleh tujuh skenario yang dipilih, definisi bidang yang dibandingkan, interval polling, batas waktu, dan kriteria kesehatan. R1 mengukur kesetaraan bidang terpilih terhadap acuan, bukan kebenaran semantik atau keamanan konfigurasi. Pengamatan polling memberikan waktu deteksi pada resolusi instrumen, bukan waktu kejadian kontinu yang persis. R3 tidak diukur karena audit API server tidak tersedia, sehingga klaim keterlacakan tidak dapat dibuat. Skenario A, F, dan G tidak dapat digeneralisasi sebagai ukuran *self-healing* untuk sumber daya terkelola.

3.9.2 Validitas Internal

Validitas internal dapat terancam oleh perbedaan beban klaster dari waktu ke waktu, perubahan konfigurasi selain intervensi, ketidaksetaraan inventaris, serta perbedaan jadwal rekonsiliasi. Karena kedua mode berurutan pada satu stack, tidak ada isolasi penuh: beban latar dan keadaan klaster dapat berubah antarblok. Urutan pengujian yang diacak, nilai dasar yang dibekukan, pencatatan keadaan, penjedaan/pemulihan ArgoCD, dan gerbang kesiapan dirancang untuk mengurangi ancaman tersebut, tetapi tidak menjamin bahwa satu-satunya perbedaan adalah paradigma abstrak. Perbandingan tetap mengikutsertakan perbedaan implementasi yang melekat pada pemeriksa/korektor `kubectl` dan ArgoCD.

3.9.3 Validitas Eksternal

Validitas eksternal dibatasi pada satu klaster, satu stack InvenioRDM produksi-like, satu beban uji formal multi-service InvenioRDM, satu alat GitOps, serta konfigurasi dan versi yang benar-benar digunakan. Studi ini adalah studi kasus tunggal; hasilnya tidak dapat digeneralisasi ke semua aplikasi, klaster, atau alat GitOps. Pilot S1/S2 historis/offline tidak memperluas cakupan inferensi.

3.9.4 Validitas Kesimpulan

Jumlah pengulangan per sel terbatas (15 run; 10 untuk R2 dan 5 untuk R1) dan tidak dijamin memiliki daya statistik per skenario. Uji nonparametrik, ukuran efek, interval kepercayaan, dan pelaporan penyensoran dipakai agar kesimpulan tidak bergantung pada asumsi normalitas atau independensi yang tidak terpenuhi. Kegagalan, data tersensor, dan ketidakpastian dilaporkan; tidak ada klaim ekivalensi atau superioritas universal. Reliabilitas bergantung pada konsistensi kolektor, sinkronisasi jam, cakupan log, dan kemampuan mengulang reset. Sampai gerbang tersebut dipenuhi, hasil eksperimen belum tersedia dan protokol tetap berstatus draf.

BAB IV
HASIL DAN ANALISIS

4.1 Status Data dan Provenans

Pada draf ini belum tersedia dataset eksperimen formal yang tervalidasi. Apparatus belum diuji terhadap keseluruhan protokol pada Bab III dan protokol masih menunggu tinjauan pembimbing. Pilot terbatas pada stack yang sama bukan data formal. Oleh karena itu, bagian ini mendefinisikan informasi yang perlu dilaporkan, tetapi tidak menyajikan angka, uji statistik, ukuran efek, atau kesimpulan komparatif.

Sebelum pengisian hasil, dokumentasikan konfigurasi klaster yang benar-benar digunakan, versi Kubernetes dan ArgoCD, cakupan aplikasi, kebijakan *auto-sync*/*self-heal*/*prune*, manifest serta inventaris formal, revisi sumber, SHA-256 hasil render, image digest, interval polling aktual, sinkronisasi jam, dan kriteria kesiapan. Nilai yang pernah disebut sebagai target pada draf sebelumnya bukan hasil verifikasi. Laporkan pula setiap penyimpangan protokol, kegagalan kesiapan, dan perubahan apparatus.

Pengujian formal direncanakan pada satu stack *workload* multi-service InvenioRDM produksi-like di namespace `invenio`; tidak ada stack kembar `exp-*` dan tidak ada eksekusi paralel A/B. Surrogat S1/S2 historis/offline tidak dipool atau ditafsirkan sebagai hasil formal. Identitas sensitif dan seluruh nilai Secret harus disanitasi dari log, tabel, gambar, serta lampiran.

4.2 Format Pelaporan E1: Kesetaraan dan Kesehatan (R1)

Kesetaraan inventaris dan bidang yang ditetapkan dilaporkan terpisah dari kesehatan layanan. Evaluator yang sama digunakan pada kedua mode. Tabel 4.1 merupakan skema pelaporan, bukan hasil; isinya menunggu pengumpulan data formal.

Tabel 4.1: Skema hasil R1 per skenario dan mode

Skenario	Mode	Kesetaraan konfigurasi: dasar / pascainjeksi / pascapemulihan	Kesehatan layanan pada pengamatan terkait
B–E	imperatif / GitOps	Belum tersedia	Belum tersedia
A	HPA / HPA	Kontrol HPA; replika bukan kesetaraan statis	Belum tersedia
F	imperatif / GitOps	Tidak termasuk hasil utama R1; dicatat sebagai batas cakupan	Dilaporkan terpisah jika relevan
G	imperatif / GitOps	Deskriptif; bukan *self-heal* ArgoCD	Dilaporkan terpisah jika relevan

4.3 Format Pelaporan E2: Waktu (R2)

Untuk setiap skenario dan mode, hasil kelak mencantumkan target run, jumlah injeksi terverifikasi, jumlah waktu teramati dan tersensor, serta durasi deteksi ($t_d - t_0$), ketersediaan bukti ($t_e - t_0$), dan pemulihan konfigurasi penuh ($t_r - t_0$). Nilai waktu dihitung dari cap waktu kolektor dan tidak diinterpolasi melewati jeda polling.

Tabel 4.2: Skema hasil R2 per skenario dan mode

Skenario	Mode	Run valid / tersensor dan alasan	Distribusi durasi: deteksi / bukti / pemulihan
B–E	imperatif / GitOps	Belum tersedia	Belum tersedia
A	HPA / HPA	Dilaporkan terpisah	Respons HPA, bukan efikasi ArgoCD
F	imperatif / GitOps	Dilaporkan sebagai pengamatan batas visibilitas	Tidak menjadi hasil pemulihan drift terkelola
G	imperatif / GitOps	Dilaporkan sebagai kasus deskriptif	Bukan hasil *self-heal* ArgoCD

Run tanpa deteksi, bukti, atau pemulihan sampai batas pengamatan akan tetap tampak sebagai tersensor kanan dan disertai jumlahnya. Kegagalan injeksi dan kegagalan instrumentasi dicatat terpisah dari kejadian mekanisme tidak mendeteksi. Run dengan pemulihan tidak lengkap tidak dihilangkan hanya untuk membentuk distribusi waktu yang lebih menguntungkan.

4.4 R3: Tidak Diukur

R3 (bukti keterlacakan) tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi untuk kedua mode. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik. Bagian ini tidak menyediakan rubrik hasil R3; keterlacakan dibahas sebagai keterbatasan pada Bab V dan sebagai pekerjaan lanjutan setelah audit diaktifkan.

4.5 Rencana Analisis Komparatif

Ringkasan dan perbandingan dilakukan per skenario, bukan dengan menggabungkan semua intervensi menjadi satu angka. Sampel formal adalah 15 pengulangan per skenario per mode: 10 pengulangan protokol R2 dan 5 pengulangan protokol R1, sehingga totalnya $7 \times 2 \times 15 = 210$ run pada satu kasus. Angka tersebut merupakan target yang direncanakan dan belum dibuktikan cukup untuk mendeteksi efek pada setiap skenario (Dybå et al., 2006; Kitchenham and Madeyski, 2024). Statistik deskriptif, jumlah run valid, kejadian tersensor, serta ukuran efek yang sesuai harus dilaporkan bersama hasil.

Karena mode berurutan pada stack yang sama, observasi tidak diperlakukan independen secara naif. Uji nonparametrik berpasangan/blok (misalnya Wilcoxon *signed-rank*) hanya dapat dipertimbangkan untuk hasil waktu yang teramati lengkap dan dapat dipasangkan. Jika pemasangan tidak dapat dipertahankan, *Mann–Whitney U* dua sisi dilaporkan sebagai eksploratif. Ukuran efek (Cliff's δ atau Hodges–Lehmann) disertai interval kepercayaan bootstrap. Waktu deteksi dan pemulihan yang tersensor kanan dianalisis dengan Kaplan–Meier serta median/*restricted mean survival*; metode dan keluarga uji harus dibakukan sebelum melihat hasil atau hasil dilaporkan deskriptif. Tidak ada nilai p, ukuran efek, ataupun interpretasi arah perbedaan yang dapat dituliskan sebelum pengumpulan serta analisis data formal.

4.6 Batas Interpretasi

RQ1 hanya dapat dijawab terhadap inventaris dan bidang yang ditetapkan; kesetaraan bukan bukti bahwa konfigurasi benar secara semantik. RQ2 membedakan sinyal deteksi, tersedianya bukti yang dapat diverifikasi, dan kembalinya konfigurasi ke keadaan yang diinginkan; kesehatan layanan tetap dilaporkan sebagai dimensi

terpisah. R3 tidak diukur.

Skenario A mempertahankan HPA pada kedua mode; respons replika bukan keberhasilan ArgoCD memulihkan bidang yang diabaikan. F merupakan pengujian batas visibilitas sumber daya tak terkelola, bukan janji prune/self-heal. G bukan kasus *self-heal* ArgoCD dan dianalisis deskriptif. A, F, dan G dilaporkan tersendiri, tidak digabungkan ke efikasi B–E. Tidak adanya OutOfSync bukan deteksi nol.

Karena kedua mode memakai satu stack produksi-like secara berurutan, perlakuan tidak terisolasi dari beban latar dan dependensi bersama. Sampai data formal tersedia, tidak dapat dinyatakan mode mana yang lebih konsisten, lebih cepat, atau lebih terlacak. Kondisi ini merupakan batas bukti draf, bukan hasil eksperimen negatif dan bukan bukti bahwa instrumentasi gagal.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Naskah ini telah merumuskan rancangan studi kasus komparatif terkontrol untuk membandingkan mode imperatif terotomasi berbasis `kubectl` dengan mode GitOps berbasis ArgoCD pada satu stack *workload* multi-service InvenioRDM produksi-like. Rancangan membedakan kesetaraan konfigurasi dari kesehatan layanan, memisahkan waktu deteksi, ketersediaan bukti, dan pemulihan konfigurasi penuh, serta menetapkan mode imperatif sebagai penjedaan ArgoCD pada stack yang sama, bukan *deployment* terpisah.

Kesimpulan empiris untuk setiap pertanyaan penelitian belum dapat ditarik:

1. **RQ1:** Perbedaan kesetaraan konfigurasi dan kesehatan layanan belum diketahui karena data formal belum tersedia.

2. **RQ2:** Perbedaan waktu deteksi, ketersediaan bukti penyimpangan, dan pemulihan konfigurasi belum diketahui. Tidak ada hasil waktu yang dapat diisi dengan asumsi atau angka sintetik.

R3 (bukti keterlacakan) tidak diukur karena audit API server Kubernetes tidak tersedia. Ketiadaan audit tidak menghambat *deployment* atau pengukuran R1/R2; R3 tidak dilaporkan sebagai nol atau tidak ada perbedaan. Dengan demikian, draf ini tidak menyimpulkan efektivitas, keunggulan, atau kelayakan adopsi salah satu mode. Skenario A sebagai kontrol HPA, F sebagai batas visibilitas, dan G sebagai kasus deskriptif bukan *self-heal* ArgoCD harus dilaporkan terpisah, bukan sebagai bukti keberhasilan atau kegagalan rekonsiliasi.

5.2 Saran

Sebelum pengujian formal, protokol perlu ditinjau pembimbing, lalu apparatus harus divalidasi terhadap definisi penerimaan injeksi, skema inventaris dan bidang, keselamatan pencatatan Secret, interval sampling satu detik, timestamp, kriteria kesehatan, batas waktu, prosedur penjedaan/pemulihan ArgoCD, serta mekanisme reset. Sampel formal perlu dilaksanakan sesuai rencana 15 pengulangan per skenario per

mode (10 R2 dan 5 R1; total 210 run) dengan pembekuan revisi sumber, SHA-256 render, image digest, dan versi runtime sebelum run pertama. Surrogat S1/S2 historis/offline tidak boleh menggantikan beban formal.

Pengumpulan data berikutnya sebaiknya mencatat konfigurasi serta versi yang benar-benar digunakan, mempertahankan semua kategori kegagalan dan penyensoran, melaporkan hasil per skenario, dan menyimpan bukti yang telah disanitasi. Klaim komparatif perlu dibatasi pada prosedur, *workload*, klaster, dan resolusi instrumen yang diuji. Pekerjaan lanjutan dapat mengaktifkan audit API server untuk mengukur R3, menguji *workload* atau ukuran klaster lain, serta membandingkan implementasi GitOps lain dengan definisi metrik yang sama.

5.3 Keterbatasan Penelitian

Keterbatasan yang melekat pada rancangan ini meliputi penggunaan satu klaster, satu stack InvenioRDM produksi-like, dan satu implementasi GitOps. Kedua mode dijalankan berurutan pada stack yang sama; tidak ada stack kembar `exp-*`, sehingga perlakuan tidak terisolasi dari beban latar, dependensi bersama, dan keadaan klaster yang berubah. Studi ini adalah studi kasus tunggal; hasilnya tidak otomatis berlaku bagi konteks lain.

Mode imperatif yang diuji adalah satu otomasi `kubectl` tertentu, sehingga tidak mewakili semua cara kerja imperatif. Interval sampling satu detik membatasi ketelitian waktu kejadian, dan target 210 run bukan jaminan daya statistik per skenario. Skenario A adalah kontrol HPA; F berada di luar cakupan sumber daya terkelola dan hanya menguji batas visibilitas; G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. Ketiganya tidak dipool ke estimasi efikasi B–E.

R3 tidak diukur karena audit API server Kubernetes tidak tersedia; tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi. Pengukuran kesetaraan juga tidak menilai apakah konfigurasi yang diinginkan aman atau benar secara semantik. Selain itu, versi apparatus, inventaris formal, batas waktu pengamatan, serta kriteria kesehatan belum diverifikasi dalam draf ini. Protokol juga belum memperoleh tinjauan pembimbing dan kemampuan apparatus untuk menjalankannya belum divalidasi. Seluruh keterbatasan tersebut perlu dipertimbangkan sebelum hasil empiris tersedia atau rekomendasi praktis dirumuskan.

DAFTAR PUSTAKA

R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.

Argo Project. Argocd documentation, 2024. URL `https://argo-cd.readthedocs.io`.

Argo Project. Argo cd documentation: Automated sync policy. `https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/`, 2025. Dokumentasi resmi perangkat lunak.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.

CNCF GitOps Working Group. Opengitops principles v1.0.0. `https://opengitops.dev`, 2021.

Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.

M. D'Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master's thesis, Politecnico di Torino, 2021.

Tore Dybå, Vigdis By Kampenes, and Dag I. K. Sjøberg. A systematic review of statistical power in software engineering experiments. *Information and Software Technology*, 48(8):745–755, 2006. doi: 10.1016/j.infsof.2005.08.009.

Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

Barbara Kitchenham and Lech Madeyski. Recommendations for analysing and meta-analysing small sample size software engineering experiments. *Empirical Software Engineering*, 29(6), 2024. doi: 10.1007/s10664-024-10504-1.

Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

Gerardo Lopez and Saloni Narang. Gitops in 2025: From old-school updates to the modern way. `https://www.cncf.io/blog/2025/06/09/gitops-in-2025-from-old-school-updates-to-the-modern-way/`, 2025. Artikel industri CNCF, bukan artikel peer-review; hanya untuk konteks.

M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.

E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.

O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.

A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.

Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.

D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.

R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.

H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.

Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 52(3):1125–1144, 2026. doi: 10.1109/TSE.2026.3659785.

LAMPIRAN A

LAMPIRAN A

Lampiran ini disiapkan untuk artefak yang akan dipublikasikan setelah protokol ditinjau, apparatus divalidasi, dan pengumpulan data formal dilakukan. Pada draf ini, skrip, konfigurasi aktual, dan dataset tervalidasi belum dilampirkan. Ketiadaan artefak tersebut tidak menunjukkan bahwa pengujian telah berhasil atau gagal.

1.1 Artefak Eksperimen yang Direncanakan

Lampiran versi akhir dapat memuat sumber pemeriksa dan korektor imperatif berbasis `kubectl`, konfigurasi aplikasi ArgoCD, manifest dan inventaris workload formal, skema kolektor, serta prosedur pemulihan nilai dasar. Setiap artefak perlu diberi versi dan dikaitkan dengan konfigurasi aktual eksperimen. Materi pilot S1/S2 harus dipisahkan dari artefak dan data pengujian formal.

1.2 Skema Data dan Perlindungan Informasi

Skema log formal perlu menyimpan pengenal run, lingkungan, skenario pada berkas metadata terpisah dari masukan pengendali, status penerimaan injeksi, cap waktu monotonik dalam detik dan UTC untuk t_0, t_a, t_d, t_e, dan t_r, interval polling aktual, hasil kesetaraan dan kesehatan, status penyensoran, kesalahan, serta alasan run tidak valid. t_a menyimpan waktu penerimaan respons API injeksi. Metadata pengenal skenario tidak boleh diberikan kepada mekanisme pendeteksi atau korektor.

Snapshot dan bukti harus disanitasi sebelum disimpan atau dibagikan. Nilai Secret tidak dimasukkan ke listing, tangkapan layar, log, atau dataset publik. Pencocokan Secret dilakukan di memori; artefak yang dibagikan hanya memuat identitas bidang dan hasil kesetaraan, tanpa nilai rahasia atau hash yang dapat dipakai untuk menebak isinya.

1.3 Status Lampiran

Seluruh item pada lampiran ini masih berupa kebutuhan artefak, bukan skrip atau konfigurasi yang telah diverifikasi. Isi lampiran hanya dapat dilengkapi berdasarkan bukti yang benar-benar dihasilkan oleh eksperimen dan telah ditinjau untuk

mencegah kebocoran informasi sensitif.